Formation Développeurs .NET ▪ Ingénieurs DevOps

Claude Code pour les Dev & DevOps

Piloter un agent de coding en environnement bancaire

4 jours ▪ 9 modules + 1 annexe ▪ 11 labs ▪ 1 capstone

Bassem Ben Hamed ▪ ODDO BHF ▪ 15 octobre 2026

Pourquoi cette formation

Le constat

- L'IA générative est arrivée dans les équipes : chacun a **ses** prompts, ses astuces, ses copier-coller.
- Rien n'est partagé, rien n'est tracé, rien n'est **imposé**.
- Or un agent de coding **agit** : il écrit du code, lance `dotnet`, `docker`, `kubectl`.

Le parti pris

- On n'apprend pas « à prompter » : on apprend à **outiller** et à **cadrer** un agent.
- Chaque brique est posée **quand la précédente atteint sa limite**.
- Chaque garde-fou est **testé**, pas décrit.

À retenir

L'objectif n'est pas de coder plus vite. C'est de transformer vos **façons de travailler** en actifs exécutables, versionnés et partagés : une convention, une revue, un garde-fou, une procédure.

À qui s'adresse la formation

Deux publics, un parcours

- **Dev** : développeurs **C# / .NET** — API, EF Core, Blazor, tests.
- **DevOps** : ingénieurs **DevOps** — CI/CD, conteneurs, Kubernetes, observabilité.
- **Débutants acceptés** sur Claude Code et sur l'IA générative.

Pré-requis réels

- À l'aise avec un terminal et Git.
- Au moins l'un de : .NET, Docker, Kubernetes.
- Un compte Claude (Pro, Max, Team, Enterprise ou Console).

Format

≈ **30 % de théorie** / **70 % de pratique**, en direct dans un terminal, sur une application microservices réelle. Chaque module ouvre sur un cas **Dev et** un cas **DevOps** ; deux labs ont un parcours par métier ; le capstone se fait en **binôme Dev + DevOps**.

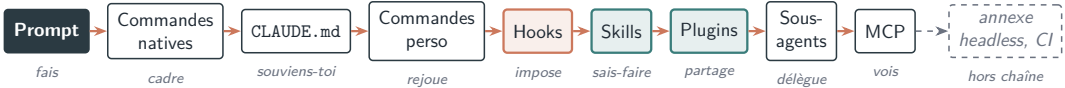
Ce que vous saurez faire à la fin

#	Compétence	Validée par
1	Piloter et cadrer une session : `/plan`, `/rewind`, `/context`, `/code-review`	Module 01
2	Donner au projet une **mémoire vérifiable** : `CLAUDE.md`, préfixes `@ ! #`	Module 02 ▪ Lab 01
3	Capitaliser les prompts d'équipe en **commandes**	Module 03
4	Imposer des **garde-fous déterministes** : secrets, destructif, migrations, tests rouges, audit	Module 04 ▪ Labs 02-03
5	Écrire des **skills** avec fichiers d'appui, et contrôler qui les déclenche	Module 05 ▪ Lab 04
6	**Consommer, produire, distribuer, gouverner** des plugins	Module 06
7	Déléguer à des **sous-agents** indépendants, restreints, en parallèle	Module 07 ▪ Labs 08, 11
8	Brancher l'agent sur la base, le navigateur, la forge, la télémétrie **en lecture seule** (MCP)	Module 08 ▪ Labs 04, 05, 10
9	Faire tourner l'agent **sans humain** en CI, sans lui laisser de décision	Annexe ▪ Lab 06
10	Livrer une fonctionnalité **de la migration au cluster observé**, et tenir un incident	Capstone

La progression pédagogique



Le fil conducteur : chaque brique lève une limite



L'histoire d'une équipe qui adopte l'outil

On **demande**, on **cadre**, on **mémorise**, on **rejoue**, on **impose**, on **capitalise**, on **diffuse**, on **délègue**, puis on **connecte**. **La chaîne s'arrête au MCP.**

Pourquoi les hooks avant les skills

Dès qu'on rejoue un prompt, on veut une **garantie** (« jamais de secret committé ») avant un **savoir-faire**. Le contrôle d'abord, la procédure ensuite.

À retenir

On ne passe à la brique suivante qu'après avoir **senti la limite** de la précédente sur un cas réel. Chaque module s'ouvre sur cette limite et se termine sur la suivante.

Chaque passage est justifié par un cas d'usage (1/2)

Vers…	La limite rencontrée	Cas Dev	Cas DevOps
Commandes natives	Je ne sais ni l'arrêter, ni annuler, ni voir ce qu'il a en tête.	Un refactoring dérape : `/rewind`, pas « annule ».	`/plan` avant de toucher au `docker-compose.yml`.
`CLAUDE.md`	Chaque session repart de zéro.	Il code `localhost:5147` en dur au lieu du service discovery.	Il lance `dotnet build` au lieu de `ECommerce.slnx`.
Commande perso	Je retape le même prompt de vingt lignes.	`/review-diff` : la checklist de revue d'équipe.	`/ship` : tests, commit, PR, toujours dans le même ordre.
Hook	Ma commande *dit* « pas de secret » ; rien ne l'*empêche*.	Un mot de passe d'`appsettings` part dans un commit.	Un `docker compose down -v` efface la base d'un collègue.
Skill	Un hook sait dire **non**, pas **faire** une procédure.	Ajouter une colonne : modèle, migration, **relecture du SQL**, tests.	Conteneuriser : multi-stage, non-root, scan, compose.

Chaque passage est justifié par un cas d'usage (2/2)

Vers…	La limite rencontrée	Cas Dev	Cas DevOps
Plugin	Tout vit dans **mon** `.claude/` ; douze collègues, huit dépôts.	Installer `csharp-lsp` plutôt que bricoler un hook de build.	Les mêmes garde-fous pour toute l'équipe, versionnés.
Sous-agent	La revue sature ma session et valide par complaisance.	Une revue sécurité **indépendante**, sans écriture possible.	Un pod en `CrashLoopBackOff` : 800 lignes lues ailleurs, 15 rendues.
MCP	Il ne voit que le disque et le shell.	Le schéma PostgreSQL réellement migré ; la page dans Chrome.	Les traces via Aspire ; Grafana pendant un incident.

Le document de référence

`docs/choisir-sa-brique.md` : toutes les transitions, l'**arbre de décision** et les confusions qui coûtent cher.

Hors chaîne : l'agent sans terminal

`claude -p`, `/loop`, la CI : pas une brique de plus, mais **les mêmes briques, exécutées autrement**. En annexe, puis au Lab 06.

Les 9 modules — la limite que chacun lève

#	Module	La limite qui le justifie	Durée
00	Prise en main	« Que voit-il, que fait-il sans me demander ? »	45 min
01	Commandes natives	« Je ne sais ni l'arrêter, ni annuler, ni voir ce qu'il a en tête. »	45 min
02	Contexte & mémoire	« Il ignore nos conventions à chaque session. »	40 min
03	Commandes personnalisées	« Je retape le même prompt de revue tous les jours. »	30 min
04	Hooks	« Ma commande *dit* de ne pas committer de secret ; rien ne l'*empêche*. »	50 min
05	Skills	« Un hook sait dire non, pas *faire* une procédure. »	45 min
06	Plugins	« Douze collègues, huit dépôts — et je réinvente ce qui existe. »	60 min
07	Sous-agents	« La revue sature ma session et valide par complaisance. »	45 min
08	MCP	« Il ne voit ni la base, ni le navigateur, ni les traces. »	60 min
A	*Annexe — l'agent sans terminal*	*Hors chaîne : un mode d'exécution (`claude -p`, `/loop`), pas une brique*	40 min

Canevas commun : **La limite** ▪ **Le concept** ▪ **Anatomie** ▪ **Mini-lab** ▪ **Erreurs fréquentes** ▪ **Checklist** ▪ **La limite suivante**

Les 11 labs — vos cas d'usage

Bloc 1 — Code, qualité & sécurité

01 Git & GitHub : un correctif jusqu'à la PR
02 Tests d'intégration, faux service Catalog
03 Sécurité & secrets : poste, dépendances, CI

Bloc 2 — Données & interface (nouveaux)

04 **Créer la base PostgreSQL** : schéma, migration relue, Testcontainers, MCP en lecture seule
05 **Améliorer le frontend** : filtres, états, accessibilité, vérifié dans Chrome

Bloc 3 — Livraison

06 CI/CD avec revue headless
07 Conteneurs, avec PostgreSQL
08 Kubernetes : app + base, Secret, pannes
09 Helm multi-environnements

Bloc 4 — Run, deux parcours

10 Observabilité — **Dev** télémétrie + MCP Aspire / **DevOps** Grafana + MCP
11 Incident — **Dev** contrat cassé, `git bisect` / **DevOps** rotation de Secret

Capstone

Une fonctionnalité de la migration au cluster observé, en binôme, avec épreuve croisée.

Déroulé des 4 jours



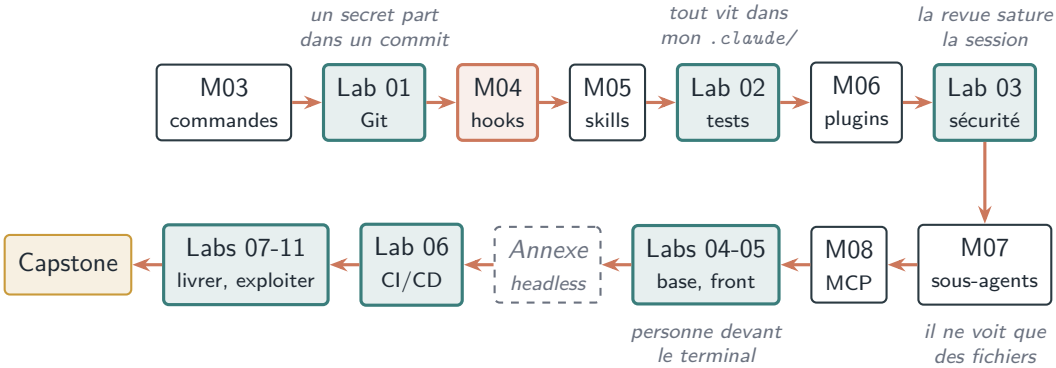
Programme

Jour	Matin	Après-midi
J1 *Fondations & prise en main*	Fondations IA générative TP prompt engineering (13 techniques)	Modules **00** → **03** session, commandes natives, mémoire, commandes perso
J2 *Imposer & capitaliser*	Lab **01** (Git) Module **04** (hooks) Module **05** (skills)	Lab **02** (tests) Module **06** (plugins) Lab **03** (sécurité)
J3 *Déléguer & connecter*	Modules **07** (sous-agents) et **08** (MCP) Lab **04** (PostgreSQL)	Lab **05** (frontend) **Annexe** : l'agent sans terminal Lab **06** (CI/CD)
J4 *Livrer & exploiter*	Labs **07** (Docker), **08** (Kubernetes), **09** (Helm)	Labs **10** (observabilité), **11** (incident) **Capstone** et restitution

À retenir

Modules et labs **alternent** : chaque lab applique les briques vues et révèle la limite de la suivante. Variables d'ajustement : Lab **09** (Helm) et le parcours DevOps du Lab **10**. Jamais le Lab **03** ni les garde-fous.

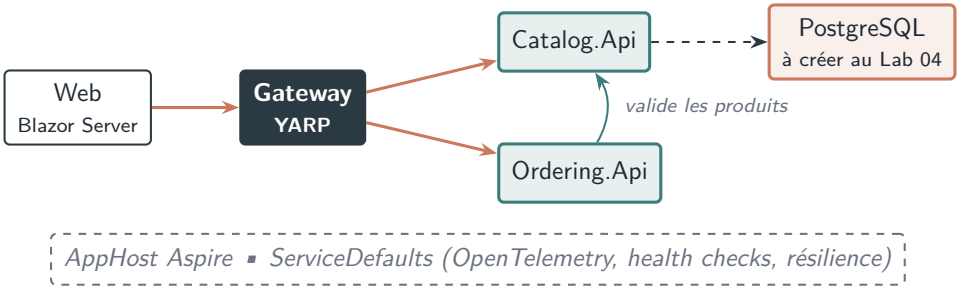
L'alternance : chaque lab révèle la limite suivante



Le lab comme révélateur

Chaque lab se termine par **La limite révélée** : le cas d'usage qu'il ne résout pas, et le module qui le traite. Aucune brique n'est présentée avant que vous en ayez eu besoin.

L'application fil rouge



Ce que c'est

Microservices **.NET 10 / ASP.NET Core**, minimal APIs et EF Core, orchestrés par **.NET Aspire**. Les services se trouvent par **service discovery** : aucune URL en dur.

Livrée volontairement nue

Ni `.claude/`, ni tests, ni base réelle, ni `Dockerfile`, ni CI. **C'est vous qui les produisez**, lab après lab.

Le cadre : garde-fous & conformité



Le principe qui structure toute la formation

Un garde-fou qui dépend du modèle n'est pas un garde-fou.

Niveau	Mécanisme	Fiabilité	Usage
1	Consigne dans le prompt	Aléatoire	Une demande ponctuelle
2	Règle dans `CLAUDE.md`, skill	Bonne, non garantie	Conventions, procédures
3	Règle `/permissions`	Appliquée par le harnais	`allow` / `ask` / `deny` par outil
4	**Hook**	**Déterministe**	**Anti-secret, destructif, audit**
5	Contrôle de plateforme	Hors périmètre de l'agent	Branche et environnement protégés, RBAC

À retenir

Plus l'action est irréversible, plus le niveau doit être élevé. Une convention de nommage vit dans `CLAUDE.md` ; un secret se bloque par hook ; une mise en production se protège côté plateforme.

Les six règles du cadre bancaire

	Règle	Mise en œuvre technique
R1	Aucune donnée sensible (prompt, fichier, log)	Hooks anti-secret (écriture + commit), `deny` en lecture, `user-secrets`, télémétrie sans PII
R2	Aucune action destructive sans validation	Hook `guard-destructive` + `deny`
R3	Aucun déploiement en production automatique	Environnement protégé côté GitHub
R4	Schéma, pipeline, infra relus par un humain	Skill `ef-migration`, `db-reviewer`, hook `guard-migrations`, rendu Helm
R5	Traçabilité des actions de l'agent	Hook `audit-log`, journal hors dépôt (DORA)
R6	Une donnée externe n'est pas une instruction	MCP en lecture seule : rôle SQL, jeton fine-grained, *Viewer*

Le principe qui prime sur tous les autres

L'IA propose, l'ingénieur décide et vérifie. Un hook n'est pas un modèle : il s'exécute toujours.

Ce qu'on ne fait pas pendant ces 4 jours

Hors cadre

- Aucune connexion à un **système de production**.
- Aucun **jeton d'entreprise** : comptes et dépôts de test.
- Aucune **donnée réelle** : données fictives uniquement.
- Aucun **agent autonome** qui merge, migre ou déploie.

Pas au programme

Les labs **Multipass**, **Terraform** et **Ansible** relèvent du parcours Ops. Ici, l'observabilité et l'incident sont **adaptés aux deux métiers**.

Et pourtant, tout est réel

La base tourne, les migrations s'appliquent, les images sont scannées, les pods plantent pour de vrai — et les hooks bloquent réellement vos commandes. **Aucune simulation.**

Pratique



Vérifier son poste — maintenant

```
# 1. Claude Code
curl -fsSL https://claude.ai/install.sh | bash
claude --version && claude doctor          # >= 2.1

# 2. L'outillage de la formation
./docs/check-prereqs.sh

# 3. L'application fil rouge, sur une copie de travail
export FORMATION="$PWD"
cp -r ecommerce-app ~/lab-ecommerce && cd ~/lab-ecommerce
dotnet build ECommerce.slnx
```

Indispensable

`claude`, `git`, `gh`, `dotnet` (SDK 10), `jq`, `docker`

Selon les labs

`dotnet-ef`, `node`/`npx`, `gitleaks`, `trivy`, `kubectl`, `k3d`, `helm`, `aspire`, `csharp-ls`, Chrome + *Claude in Chrome*

Livrables de la formation

Pour vous

- Le dépôt complet : modules, labs, corrigés cumulatifs.
- `choisir-sa-brique.md`, la **cheat-sheet**, le **glossaire**.
- Le document **garde-fous & conformité**.
- Votre **bibliothèque de prompts** (TP du J1).

Pour votre équipe

- Le plugin `oddo-dev-toolkit` : 10 skills, 6 sous-agents, 7 hooks.
- Le bundle `oddo-dev-standard` : le **poste standard** en une commande.
- Le **marketplace interne** qui les distribue.
- Votre plan **30 / 60 / 90 jours**.

À retenir

Le vrai livrable n'est pas ce dépôt : c'est ce que vous mettrez en place **la semaine prochaine**, sur un dépôt réel.

On commence.

Fondations IA générative & prompt engineering